

01

컴퓨터 비전과 OpenCV

이미지를 숫자로 표현하고 의미 있는 정보를 추출하는 기술

컴퓨터 비전(Computer Vision)은 이미지와 영상에서 객체, 문자, 움직임, 위치 등의 정보를 추출하여 컴퓨터가 장면을 이해하도록 만드는 인공지능 분야이다. **OpenCV**는 이미지 처리, 특징 추출, 객체 검출, 카메라 처리 등을 제공하는 오픈소스 라이브러리이며 Python에서는 **cv2** 모듈로 사용한다.

대표 활용 분야

분야	활용 사례
제조	제품 불량 검사, 부품 위치 확인
의료	X-ray·CT 영상 전처리와 병변 후보 탐색
교통	차량·보행자·차선 검출
문서	OCR 전처리, 영수증·계약서 분석
보안	얼굴 인식, CCTV 객체 추적

설치와 버전 확인

```
pip install opencv-python numpy matplotlib

import cv2
print(cv2.__version__)
```

기본 처리 흐름

입력(imread, VideoCapture) → 전처리(resize, cvtColor, blur) → 분석(threshold, Canny, findContours) → 시각화·저장(imshow, imwrite)

학습 목표

→ 이미지 배열 구조와 BGR 색상 순서를 설명한다. → 핵심 영상 처리 함수를 조합한다. → 카메라 영상을 프레임 단위로 처리한다.

02

이미지 입출력과 배열 구조

OpenCV 이미지는 높이·너비·채널로 구성된 NumPy 배열이다

컬러 이미지는 일반적으로 **(height, width, channels)** 형태의 3차원 배열이며 픽셀 값은 0~255 범위의 uint8 자료형을 사용한다. OpenCV의 컬러 채널 순서는 RGB가 아니라 **BGR**이다.

표현	예시	설명
컬러 shape	(480, 640, 3)	높이 480, 너비 640, B·G·R 3채널
그레이 shape	(480, 640)	밝기 정보만 가진 1채널
픽셀 접근	img[y, x]	배열에서는 y(행)가 먼저
관심 영역	img[y1:y2, x1:x2]	NumPy 슬라이싱으로 ROI 추출

이미지 읽기·검사·저장

```
import cv2

img = cv2.imread("sample.jpg", cv2.IMREAD_COLOR)
if img is None:
    raise FileNotFoundError("sample.jpg ■■■ ■■■■■.")

print(img.shape) # (height, width, channels)
print(img.dtype) # uint8
b, g, r = img[100, 200]

roi = img[80:280, 120:420]
ok = cv2.imwrite("roi.jpg", roi)
print("■■■ ■■■:", ok)
```

화면 출력

```
cv2.imshow("original", img)
cv2.waitKey(0) # ■ ■■■■■ ■■
cv2.destroyAllWindows() # ■■ OpenCV ■ ■■
```

좌표 주의

그리기 함수의 좌표는 (x, y)이지만 배열의 픽셀 접근은 img[y, x]이다. x는 가로 방향, y는 세로 방향이다.

03

색상·크기·도형 처리

분석 목적에 맞게 이미지를 변환하고 결과를 표시한다

색상 공간을 바꾸면 분석에 필요한 특징을 더 쉽게 분리할 수 있다. Gray는 연산량을 줄이고 밝기 중심 분석에 적합하며, HSV는 특정 색상 범위를 추출할 때 유용하다.

변환	OpenCV 코드	용도
BGR ↔ RGB	cv2.COLOR_BGR2RGB	matplotlib 출력
BGR ↔ Gray	cv2.COLOR_BGR2GRAY	에지·임계처리
BGR ↔ HSV	cv2.COLOR_BGR2HSV	색상 마스크 생성

색상과 크기 변환

```
img = cv2.imread("sample.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

h, w = img.shape[:2]
small = cv2.resize(img, (w//2, h//2),
                    interpolation=cv2.INTER_AREA)
flip = cv2.flip(img, 1) # ■■■ ■■■
```

선·사각형·원·텍스트

```
cv2.line(img, (30, 50), (400, 50), (255, 0, 0), 3)
cv2.rectangle(img, (100, 100), (350, 300), (0, 255, 0), 2)
cv2.circle(img, (500, 220), 80, (0, 0, 255), -1)
cv2.putText(img, "OpenCV", (200, 420),
            cv2.FONT_HERSHEY_SIMPLEX, 1.2,
            (255, 255, 255), 2, cv2.LINE_AA)
```

색상 순서

OpenCV에서 (255, 0, 0)은 파랑, (0, 255, 0)은 초록, (0, 0, 255)은 빨강이다. thickness=-1이면 도형 내부를 채운다.

실습: 이미지 크기를 절반으로 축소하고 관심 영역에 사각형을 그린 뒤 “ROI” 텍스트를 표시하여 저장한다.

04

필터 · 이진화 · 윤곽선

잡음을 줄이고 객체의 경계와 영역을 찾는다

블러는 작은 노이즈를 완화하고, 임계처리는 전경과 배경을 흑백으로 분리한다. Canny는 밝기 변화가 큰 경계를 찾으며, 윤곽선은 이진 이미지에 존재하는 연결 영역의 외곽선을 점 집합으로 표현한다.

기법	대표 함수	목적
가우시안 블러	GaussianBlur	일반 잡음 완화
이진화	threshold	전경과 배경 분리
에지 검출	Canny	객체 경계 후보 생성
형태학 연산	morphologyEx	작은 잡음 제거·구멍 채우기
윤곽선	findContours	객체 수·면적·위치 측정

처리 파이프라인 예제

```
import cv2
import numpy as np

img = cv2.imread("objects.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)
_, binary = cv2.threshold(blur, 0, 255,
                          cv2.THRESH_BINARY + cv2.THRESH_OTSU)

kernel = np.ones((3, 3), np.uint8)
binary = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
contours, _ = cv2.findContours(
    binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

for cnt in contours:
    if cv2.contourArea(cnt) < 500:
        continue
    x, y, w, h = cv2.boundingRect(cnt)
    cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)

cv2.imwrite("detected.jpg", img)
```

입력 조건

findContours에는 보통 흰색 객체와 검은색 배경으로 구성된 8비트 단일 채널 이진 이미지를 입력한다.

05

웹캠 영상 처리와 종합 정리

영상은 시간 순서대로 연결된 이미지 프레임이다

VideoCapture는 카메라 또는 동영상 파일에서 프레임을 읽는다. read()가 반환하는 성공 여부를 매번 검사하고, 종료 시 release()와 destroyAllWindows()를 호출하여 자원을 정리한다.

실시간 웹캠 에지 검출

```
import cv2

cap = cv2.VideoCapture(0)
if not cap.isOpened():
    raise RuntimeError("카메라를 열 수 없습니다.")

try:
    while True:
        ok, frame = cap.read()
        if not ok:
            break

        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        edges = cv2.Canny(gray, 80, 160)
        cv2.imshow("camera", frame)
        cv2.imshow("edges", edges)

        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
finally:
    cap.release()
    cv2.destroyAllWindows()
```

핵심 함수 정리

목적	함수
이미지 입출력	imread, imshow, imwrite
색상·크기 변환	cvtColor, resize, flip
잡음·경계 처리	GaussianBlur, threshold, Canny
객체 영역 분석	findContours, contourArea, boundingRect
영상 처리	VideoCapture, read, release

학습 정리

OpenCV 문제 해결의 핵심은 입력 확인 ☑ 색상·크기 정규화 ☑ 특징 또는 마스크 생성 ☑ 잡음 제거 ☑ 객체 분석 ☑ 결과 검증의 흐름을 설계하는 것이다.

참고: OpenCV 공식 문서 <https://docs.opencv.org/> · Python 패키지 <https://pypi.org/project/opencv-python/>